

IMPLEMENT CLIENT SCRIPT & UI POLICY(INCIDENT)

Project Documentation

1. Introduction

The project "Implement Client Script & UI Policy (Incident)" is developed using the ServiceNow Incident Management module to improve Incident form behaviour, data quality, validation, and user interaction.

The solution uses UI Policies and Client Scripts to control Incident fields based on defined conditions, automatically update values, validate user input, and restrict inappropriate State changes.

2. Problem Statement

Incident records may contain incomplete, inconsistent, or incorrect information. Manual checking can affect routing, reporting, SLA compliance, and service quality. Some fields also need to be controlled depending on Incident conditions, while important values may require protection from accidental modification.

3. Objectives

- Make required fields mandatory when defined conditions occur.
- Automatically update fields when appropriate.
- Control field behaviour using UI Policy.
- Validate Incident data before saving.
- Prevent direct State changes through list editing.
- Improve Incident data integrity and user experience.
- Keep the implementation lightweight and maintainable.

4. Proposed Solution

The proposed solution uses one UI Policy with UI Policy Actions and three Client Scripts on the Incident table. Together, they provide conditional field behaviour, automatic value handling, validation, and controlled user interaction.

5: Phase 1: BRAINSTORMING & IDEATION

1. Project Idea

The project idea is to improve Incident data quality and user experience by using ServiceNow UI Policies and Client Scripts to apply conditional field behaviour and validation.

2. Key Problems Identified

- Users may enter incomplete Incident information.
- Some fields may be relevant only under particular Incident conditions.
- Users may accidentally modify information that should remain protected.
- Incorrect data can affect routing, reporting, SLA compliance, and service quality.

3. Brainstorming Ideas

Idea	Expected Benefit
Make important fields mandatory	Ensures critical information is captured
Show or hide fields based on conditions	Reduces unnecessary information
Make selected fields read-only	Protects important information
Validate user-entered values	Prevents invalid data
Display helpful messages	Guides users during data entry
React to field changes	Provides dynamic form behaviour
Use clear field dependencies	Improves data consistency
Avoid unnecessary scripting	Improves maintainability

4. Ideas Converted into Requirements

Requirement	Implementation
Critical fields become mandatory	UI Policy
Field becomes read-only under a condition	UI Policy
Field is hidden when not relevant	UI Policy
Custom validation is required	Client Script
Form responds to field changes	Client Script / UI Policy
Invalid input requires immediate feedback	Client Script

6.Phase 2 :REQUIREMENTS ANALYSIS

1.Proposed Incident Scenarios

Priority-Based Control:

When an Incident reaches a defined priority, selected information can become mandatory or read-only.

Category-Dependent Fields:

Relevant fields can be displayed and required based on the selected Incident category.

Data Validation:

Client Script can provide immediate feedback when entered information does not satisfy a business rule.

User Guidance:

The form can guide users by displaying relevant fields or messages when conditions change.

2.Functional Requirements

Requirement	Expected Behaviour
High Impact UI Policy	Create High Impact Control on Incident
Condition	Impact is 1 – High
Assignment Group	Mandatory for High Impact
Urgency	Read-only and visible for High Impact
Automatic Urgency	Set Urgency to High when Impact is High
Save Validation	Block save when High Impact + Assigned To empty
State Control	Block direct State changes through list editing
Form State Update	Allow State changes through Incident form

3.Non-Functional Requirements

- User-friendly.
- Reliable.
- Fast response.
- Improved data integrity.
- Easy to demonstrate and maintain.

- Clear and consistent configuration.
- Suitable for a short-duration micro project.

4. Software Requirements

- ServiceNow Instance – used for Incident Management, UI Policy configuration, Client Script implementation, and testing.
- Web Browser – used to access the ServiceNow instance.
- Internet Connection – required to access the ServiceNow instance.
- Administrative / Configuration Access – required to create and modify ServiceNow configurations.

5. Hardware Requirements

- Laptop for accessing ServiceNow
- Configuration
- Script creation
- Testing and documentation.
- Internet connection.

6. Expected System Behaviour

High Impact Incident

Impact = High → Assignment Group becomes Mandatory → Urgency becomes Read-only → Urgency is Automatically set to High.

Assigned To Validation

Impact = High + Assigned To = Empty → Save is Blocked → Error Message is Displayed.

State Control

Incident List → Direct State Change Blocked. Incident Form → State Change Allowed.

7. Overall Requirement Flow

User enters Incident details → Impact is changed → System checks the condition → UI Policy / Client Script applies → Fields are controlled or updated → User clicks Submit → Validation is performed → Incident is Saved / Blocked.

7.Phase 3:PROJECT DESIGN

1. Overall Design

The project is designed to improve data accuracy and consistency in the ServiceNow Incident Management module. When a user opens or updates an Incident record, the configured conditions are checked automatically.

User opens Incident Form → enters or changes Incident details → defined conditions are evaluated → UI Policy / Client Script applies the configured behaviour → validation is performed → Incident is saved only when the required conditions are satisfied.

2. UI Policy Design

Item	Design
Policy Name	High Impact Control
Table	Incident
Condition	Impact is 1 – High
Action	Assignment group becomes Mandatory
Reverse if False	True

3. UI Policy Action – Urgency

Field	Setting	Purpose
Urgency	Read-only = True	Prevent changing Urgency when Impact is High
Urgency	Visible = Unchanged	Keep the field visible on the form

4.Client Script Design

Script Type	Field / Scope	Purpose
onChange	Impact	Automatically set Urgency to High when Impact is High
onSubmit	Incident	Prevent saving when Assigned To is empty for High Impact
onCellEdit	State	Block direct State changes through list editing

5. Design Flow

Incident Form

↓

Impact Changed

↓

UI Policy + onChange Client Script

↓

Assignment Group Mandatory / Urgency Read-only / Urgency Automatically Set

↓

User Clicks Submit

↓

onSubmit Validation

↓

Save Allowed or Blocked

↓

List State Edit → onCellEdit Blocks Direct Edit

5. Configuration Logic

Component	Condition / Action
UI Policy	Impact = High
Assignment Group	Mandatory when condition is true
Urgency UI Policy Action	Read-only when condition is true
onChange	Impact High → Urgency High + information message
onSubmit	High Impact + Assigned To empty → prevent save
onCellEdit	Direct State list editing → block update

6.Design Objective

The design combines a UI Policy, a UI Policy Action, and three Client Scripts so that Incident fields respond appropriately to Impact and State changes. It provides a clear foundation for implementation and testing.

8.Phase 4– PROJECT PLANNING

1. Objectives of Project Planning

- Organise project activities in a proper sequence.
- Identify development tasks and required resources.
- Plan testing and documentation activities.
- Prepare the project for final demonstration.

2.Task Planning

Planning: Understand project requirements and required behaviours.

Development: Configure UI Policy, UI Policy Action and Client Scripts.

Testing: Test mandatory, save, reverse condition, list edit and form update behaviours.

Documentation: Prepare phase-wise documentation and screenshots.

Demonstration: Record project execution and final output.

3. Planning Stage

The project requirements and required system behaviours are studied. The required UI Policy, Client Scripts and expected Incident behaviour are identified. The output is a clear development plan.

4.Development Stage

The development work includes creating the High Impact UI Policy, Urgency UI Policy Action, onChange Client Script, onSubmit Client Script and onCellEdit Client Script.

5 .Testing Stage

Each configuration is tested to ensure that mandatory fields, automatic value changes, form submission and list editing work correctly.

6. Development Tasks

1. Create High Impact Control UI Policy on Incident table.

2. Create Urgency UI Policy Action.
3. Create onChange Client Script.
4. Create onSubmit Client Script.
5. Create onCellEdit Client Script.
6. Test the complete configuration.

7.Task Sequence

Requirements → UI Policy → UI Policy Action → Client Scripts → Testing → Documentation

8. Project Resources

ServiceNow instance, Incident module, Web browser, GitHub repository and Google Drive for demonstration video.

9 .Documentation & Demonstration

Documentation includes phase-wise project details, configuration steps, testing results and required screenshots. The demonstration shows the project execution from configuration to final output using the Incident form.

9.Phase 5:PROJECT IMPLEMENTATION / DEVELOPMENT

1. Development Process

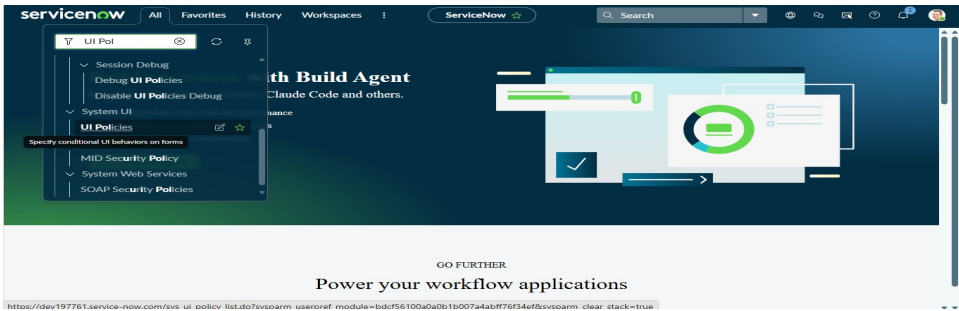
1. Configure a UI Policy for High Impact Incidents.
2. Configure a UI Policy Action to control the Urgency field.
3. Implement an onChange Client Script for automatic Urgency updates.
4. Implement an onSubmit Client Script for Incident validation.
5. Implement an onCellEdit Client Script to control State changes from the Incident list.
6. Review each configuration and prepare it for functional testing.

2. UI Policy – High Impact Control

Navigate to: System UI → UI Policies → New

Setting	Value
Name	High Impact Control
Table	Incident
Active	True
Condition	Impact is 1 – High
Assignment Group	Mandatory
Reverse if False	True

The condition ensures that the UI Policy is triggered only when the Incident Impact is set to High. Review the configuration and click Submit.



Short description	Table	Conditions	Reverse if false	On load	Updated	Order
High Impact Control	Incident [incident]	Impact=1^EQ	true	true	2026-09-23 08:48:08	100
High Impact Control	Incident [incident]	Impact=1^EQ	true	true	2026-09-23 08:35:05	100
High Impact Control	Incident [incident]	Impact=1^EQ	true	true	2026-09-23 08:26:15	100
High Impact Control	Incident [incident]	Impact=1^EQ	true	true	2026-09-23 08:21:58	100
High Impact Control	Incident [incident]	Impact=1^EQ	true	true	2026-09-23 08:07:05	100
Make assignment group mandatory	Task [task]		true	true	2026-06-12 14:20:21	100
Handle use conditions flag	Workflow Version [wf_workflow_version]	condition_type!=^EQ	true	true	2026-06-12 14:20:21	100
Show pricing implications checkbox	Question [question]	type=B^ORtype=21^EQ	true	true	2026-06-12 14:19:23	100

The screenshot shows the 'UI Policy - High Impact Control' configuration page in ServiceNow. The page is titled 'UI Policy - High Impact Control' and 'View: Advanced'. It includes a search bar and navigation tabs for 'Table', 'Incident [Incident]', and 'Application'. The 'Table' is set to 'Incident [Incident]' and the 'Application' is set to 'Global'. The 'Active' checkbox is checked. The 'Short description' is 'High Impact Control' and the 'Order' is '100'. The 'When to Apply' section has a 'Script' tab selected. The 'Conditions' section shows a condition: 'Impact' is '1 - High'. The 'Apply the UI policy actions when the form is loaded and when the user changes values on the form' checkbox is checked. The 'Reverse the effects of the UI policy actions when the Conditions evaluate to false' checkbox is checked. The 'Reverse if false' checkbox is checked. The 'Inherit' checkbox is unchecked.

The screenshot shows the 'UI Policy Action - assignment_group' configuration page in ServiceNow. The page is titled 'UI Policy Action - assignment_group' and 'View: Advanced'. It includes a search bar and navigation tabs for 'UI policy', 'High Impact Control', and 'Table'. The 'UI policy' is set to 'High Impact Control' and the 'Table' is set to 'Incident [Incident]'. The 'Field name' is 'Assignment group'. The 'Application' is set to 'Global'. The 'Mandatory' checkbox is checked. The 'Visible' dropdown is set to 'Leave alone'. The 'Read only' dropdown is set to 'False'. The 'Clear the field value' checkbox is unchecked. The 'Update' and 'Delete' buttons are visible at the bottom.

3. UI Policy Action – Urgency

7. Open the High Impact Control UI Policy.
8. Go to the UI Policy Actions related list.
9. Click New.
10. Set Field name to Urgency.
11. Set Read-only to True.
12. Leave Visible unchanged.
13. Click Submit.

When Impact is High, Urgency becomes read-only. Change Impact to another value to verify that the field can revert when the condition becomes false.

The screenshot shows the 'UI Policy Action - urgency' configuration page in ServiceNow. The page is titled 'UI Policy Action - urgency' and 'View: Advanced'. It includes a search bar and navigation tabs for 'UI policy', 'High Impact Control', and 'Table'. The 'UI policy' is set to 'High Impact Control' and the 'Table' is set to 'Incident [Incident]'. The 'Field name' is 'Urgency'. The 'Application' is set to 'Global'. The 'Mandatory' dropdown is set to 'Leave alone'. The 'Visible' dropdown is set to 'Leave alone'. The 'Read only' dropdown is set to 'True'. The 'Clear the field value' checkbox is unchecked. The 'Update' and 'Delete' buttons are visible at the bottom.

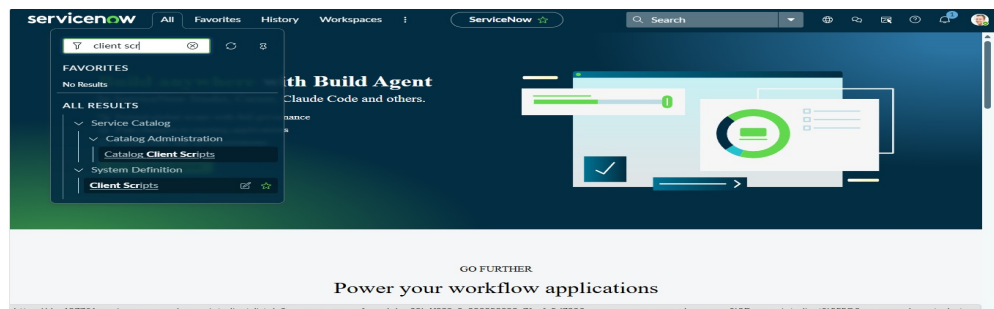
4.Client Script Implementation

onChange Client Script

Property	Value
Name	Auto set urgency for high impact
Table	Incident
Type	onChange
Field name	Impact
Active	True

Purpose: When Impact becomes High, set Urgency to High and display an informational message.

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == "") {  
        return;  
    }  
  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage(  
            'Urgency set to High for High impact incident.'  
        );  
    }  
}
```



servicenow Client Scripts Updated Search Actions on selected rows... New						
Name	Active	Table	Application	View	Type	Updated
Prevent state change via list edit	true	Incident (incident)	Global		onClientEdit	2026-09-23 12:06:41
Prevent state change via list edit	true	Incident (incident)	Global		onClientEdit	2026-09-23 12:03:00
Prevent save if Assigned To missing	true	Incident (incident)	Global		onSubmit	2026-09-23 09:54:58
Prevent save if Assigned To missing	true	Incident (incident)	Global		onSubmit	2026-09-23 09:47:49
Auto set urgency for high impact	true	Incident (incident)	Global		onChange	2026-09-23 09:20:28
Auto set urgency for high impact	true	Incident (incident)	Global		onChange	2026-09-23 09:14:18
Business Services Related List	false	Taxonomy Node [sys_taxonomy_node]	Global		onLoad	2026-06-12 14:24:45
MIC - Make fields mandatory	true	Asset [alm_asset]	Global		onChange	2026-06-12 14:17:36
EF - Show Error Prefix Format Info	true	Error Source Application [sys_error_app]	Global		onLoad	2026-06-12 14:04:01
EF - Show Error Code Format Info	true	Error Code [sys_error_code]	Global		onLoad	2026-06-12 14:04:01
EF - AutoPopulateTargetDocumentTableField	true	Error Action [sys_error_action]	Global		onLoad	2026-06-12 14:04:01
EF - Enricher Validate No IFE	true	Error Enricher [sys_error_enricher]	Global		onSubmit	2026-06-12 14:04:01
EF - Auto-populate Severity Label	true	Error Code [sys_error_code]	Global		onChange	2026-06-12 14:04:01
EF - Clear M2M Dependencies On Source App	true	Error Action Mapping [sys_error_code_to_action_map]	Global		onChange	2026-06-12 14:04:01

servicenow Client Scripts Updated Search Actions on selected rows... New

Client Script Auto set urgency for high impact

New client scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "slide.script.block.client.global" to false.

NameAuto set urgency for high impact

TableIncident (incident)

ApplicationGlobal

UI TypeDesktop

TypeonChange

Field nameImpact

Active

Inherited

Global

Description

Messages

Script

```
1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue === '') {
3     return;
4   }
5   if (newValue == "1") {
6     g_form.setValue("urgency", "3");
7     g_form.addInfoMessage("Urgency set to High for High Impact Incident.");
8   }
9 }
```

Isolate script

Update

Delete

onSubmit Client Script

Property	Value
Name	Prevent save if Assigned To missing
Table	Incident
Type	onSubmit
Active	True

Purpose: If Impact is High and Assigned To is empty, show an error and return false; otherwise allow submission.

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
      g_form.getValue('assigned_to') == '') {

    g_form.showErrorBox(
      'assigned_to',
      'Assigned To is mandatory for High impact incidents.'
    );

    return false;
  }

  return true;
}
```

The screenshot shows the ServiceNow Client Script configuration interface. The title is 'Client Script - Prevent save if Assigned To missing'. The 'Name' field is 'Prevent save if Assigned To missing'. The 'Table' is 'Incident (incident)'. The 'UI Type' is 'Desktop'. The 'Type' is 'onSubmit'. The 'Application' is 'Global'. The 'Active' checkbox is checked. The 'Inherited' checkbox is unchecked. The 'Global' checkbox is checked. The 'Description' field is empty. The 'Messages' field is empty. The 'Script' field contains the following code:

```

1 function onSubmit() {
2   if (gs_form.getValues().assigned_to == '' || gs
3     if (gs_form.getValues().assigned_to == '' || gs
4     if (gs_form.getValues().assigned_to == '' || gs
5     if (gs_form.getValues().assigned_to == '' || gs
6     if (gs_form.getValues().assigned_to == '' || gs
7     if (gs_form.getValues().assigned_to == '' || gs
8     if (gs_form.getValues().assigned_to == '' || gs
9     if (gs_form.getValues().assigned_to == '' || gs
10    if (gs_form.getValues().assigned_to == '' || gs
11    if (gs_form.getValues().assigned_to == '' || gs
12    if (gs_form.getValues().assigned_to == '' || gs
13    if (gs_form.getValues().assigned_to == '' || gs
14    if (gs_form.getValues().assigned_to == '' || gs
15    if (gs_form.getValues().assigned_to == '' || gs
16    if (gs_form.getValues().assigned_to == '' || gs
17    if (gs_form.getValues().assigned_to == '' || gs
18    if (gs_form.getValues().assigned_to == '' || gs
19    if (gs_form.getValues().assigned_to == '' || gs
20    if (gs_form.getValues().assigned_to == '' || gs
21    if (gs_form.getValues().assigned_to == '' || gs
22    if (gs_form.getValues().assigned_to == '' || gs
23    if (gs_form.getValues().assigned_to == '' || gs
24    if (gs_form.getValues().assigned_to == '' || gs
25    if (gs_form.getValues().assigned_to == '' || gs
26    if (gs_form.getValues().assigned_to == '' || gs
27    if (gs_form.getValues().assigned_to == '' || gs
28    if (gs_form.getValues().assigned_to == '' || gs
29    if (gs_form.getValues().assigned_to == '' || gs
30    if (gs_form.getValues().assigned_to == '' || gs
31    if (gs_form.getValues().assigned_to == '' || gs
32    if (gs_form.getValues().assigned_to == '' || gs
33    if (gs_form.getValues().assigned_to == '' || gs
34    if (gs_form.getValues().assigned_to == '' || gs
35    if (gs_form.getValues().assigned_to == '' || gs
36    if (gs_form.getValues().assigned_to == '' || gs
37    if (gs_form.getValues().assigned_to == '' || gs
38    if (gs_form.getValues().assigned_to == '' || gs
39    if (gs_form.getValues().assigned_to == '' || gs
40    if (gs_form.getValues().assigned_to == '' || gs
41    if (gs_form.getValues().assigned_to == '' || gs
42    if (gs_form.getValues().assigned_to == '' || gs
43    if (gs_form.getValues().assigned_to == '' || gs
44    if (gs_form.getValues().assigned_to == '' || gs
45    if (gs_form.getValues().assigned_to == '' || gs
46    if (gs_form.getValues().assigned_to == '' || gs
47    if (gs_form.getValues().assigned_to == '' || gs
48    if (gs_form.getValues().assigned_to == '' || gs
49    if (gs_form.getValues().assigned_to == '' || gs
50    if (gs_form.getValues().assigned_to == '' || gs
51    if (gs_form.getValues().assigned_to == '' || gs
52    if (gs_form.getValues().assigned_to == '' || gs
53    if (gs_form.getValues().assigned_to == '' || gs
54    if (gs_form.getValues().assigned_to == '' || gs
55    if (gs_form.getValues().assigned_to == '' || gs
56    if (gs_form.getValues().assigned_to == '' || gs
57    if (gs_form.getValues().assigned_to == '' || gs
58    if (gs_form.getValues().assigned_to == '' || gs
59    if (gs_form.getValues().assigned_to == '' || gs
60    if (gs_form.getValues().assigned_to == '' || gs
61    if (gs_form.getValues().assigned_to == '' || gs
62    if (gs_form.getValues().assigned_to == '' || gs
63    if (gs_form.getValues().assigned_to == '' || gs
64    if (gs_form.getValues().assigned_to == '' || gs
65    if (gs_form.getValues().assigned_to == '' || gs
66    if (gs_form.getValues().assigned_to == '' || gs
67    if (gs_form.getValues().assigned_to == '' || gs
68    if (gs_form.getValues().assigned_to == '' || gs
69    if (gs_form.getValues().assigned_to == '' || gs
70    if (gs_form.getValues().assigned_to == '' || gs
71    if (gs_form.getValues().assigned_to == '' || gs
72    if (gs_form.getValues().assigned_to == '' || gs
73    if (gs_form.getValues().assigned_to == '' || gs
74    if (gs_form.getValues().assigned_to == '' || gs
75    if (gs_form.getValues().assigned_to == '' || gs
76    if (gs_form.getValues().assigned_to == '' || gs
77    if (gs_form.getValues().assigned_to == '' || gs
78    if (gs_form.getValues().assigned_to == '' || gs
79    if (gs_form.getValues().assigned_to == '' || gs
80    if (gs_form.getValues().assigned_to == '' || gs
81    if (gs_form.getValues().assigned_to == '' || gs
82    if (gs_form.getValues().assigned_to == '' || gs
83    if (gs_form.getValues().assigned_to == '' || gs
84    if (gs_form.getValues().assigned_to == '' || gs
85    if (gs_form.getValues().assigned_to == '' || gs
86    if (gs_form.getValues().assigned_to == '' || gs
87    if (gs_form.getValues().assigned_to == '' || gs
88    if (gs_form.getValues().assigned_to == '' || gs
89    if (gs_form.getValues().assigned_to == '' || gs
90    if (gs_form.getValues().assigned_to == '' || gs
91    if (gs_form.getValues().assigned_to == '' || gs
92    if (gs_form.getValues().assigned_to == '' || gs
93    if (gs_form.getValues().assigned_to == '' || gs
94    if (gs_form.getValues().assigned_to == '' || gs
95    if (gs_form.getValues().assigned_to == '' || gs
96    if (gs_form.getValues().assigned_to == '' || gs
97    if (gs_form.getValues().assigned_to == '' || gs
98    if (gs_form.getValues().assigned_to == '' || gs
99    if (gs_form.getValues().assigned_to == '' || gs
100   if (gs_form.getValues().assigned_to == '' || gs

```

The 'Isolate script' checkbox is checked. The 'Update' and 'Delete' buttons are at the bottom.

onCellEdit Client Script

Property	Value
Name	Prevent state change via list edit
Table	Incident
Type	onCellEdit
Field	State
Active	True

Purpose: Alert the user and return false so direct State changes through list editing are blocked.

```

function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```

    callback(false);
}

```

The screenshot shows the ServiceNow Client Script configuration interface. The title is 'Client Script - Prevent state change via list edit'. The 'Name' field is 'Prevent state change via list edit'. The 'Table' is 'Incident (incident)'. The 'UI Type' is 'Desktop'. The 'Type' is 'onCellEdit'. The 'Field name' is 'State'. The 'Application' is 'Global'. The 'Active' checkbox is checked. The 'Inherited' checkbox is unchecked. The 'Global' checkbox is checked. The 'Description' field is empty. The 'Messages' field is empty. The 'Script' field contains the following code:

```

1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
2   alert('State cannot be updated using list
3   editing. Please open the Incident.');
```

```

    callback(false);
}

```

The 'Isolate script' checkbox is checked. The 'Update' and 'Delete' buttons are at the bottom.

5. Testing Objectives

- Verify the High Impact UI Policy.
- Verify the UI Policy Actions.
- Verify automatic Urgency update.
- Verify submission validation.
- Verify State editing restrictions.
- Ensure valid Incident records can be saved successfully.
- Identify unexpected configuration behaviour.

6. Testing Configuration

Testing is performed on the Incident form and Incident list after the UI Policy, UI Policy Action, and three Client Scripts are configured.

10. Phase 6: PROJECT TESTING

1. Test Case1 – Mandatory Enforcement

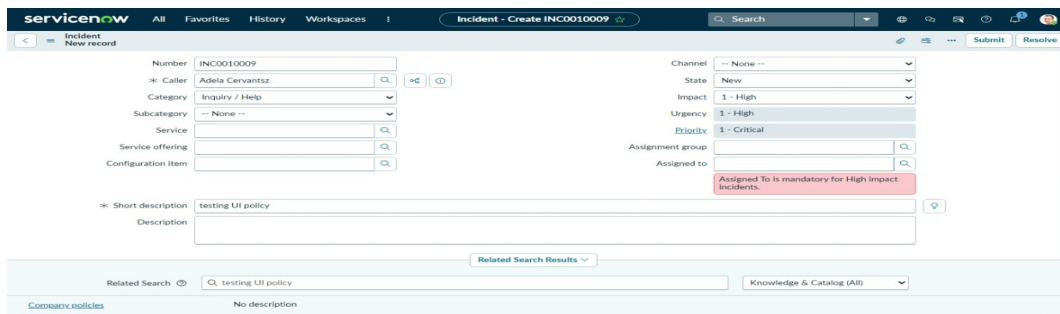
14. Go to Incident → Create New.

15. Set Impact to High.

16. Leave Assigned To empty.

17. Click Submit.

Expected Result: The Incident should not be saved because Assigned To is required when Impact is High. An error message should appear, and configured UI Policy actions should continue to work.



The screenshot shows the ServiceNow 'Incident - Create INC0010009' form. The form is partially filled with test data: Number (INC0010009), Caller (Adela Cervantes), Category (Inquiry / Help), Subcategory (None), Service (empty), Service offering (empty), Configuration item (empty), Channel (None), State (New), Impact (1 - High), Urgency (1 - High), and Priority (1 - Critical). The 'Assigned to' field is empty. A red error message is displayed below the 'Assigned to' field: 'Assigned To is mandatory for High impact incidents.' The 'Short description' field contains 'testing UI policy'. The 'Description' field is empty. The 'Related Search' section at the bottom shows a search for 'testing UI policy' with no results.

2. Test Case2 – Successful Save

18. Open the Incident form.

19. Select a user in Assigned To.

20. Click Submit.

Expected Result: The Incident should save successfully without errors. Urgency auto-setting, read-only behaviour, and State restrictions should function as expected.

3. Test Case3 – Reverse Condition

21. Open an existing Incident where Impact is High.
22. Change Impact from High to Medium.
23. Verify the field behaviour.
24. Click Submit.

Expected Result: Assignment Group should no longer be mandatory, Urgency should become editable, and the record should save successfully.

4. Test Case4 – List Edit Blocking

25. Go to Incident → All.
26. Locate an Incident.
27. Double-click the State field.
28. Try to change State directly.

Expected Result: An alert message should appear, the State change should be blocked, and the State value should remain unchanged.

servicenow									
All Favorites History Workspaces Admin									
Incidents									
Incidents									
All Active = true									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									
Incidents									
Number									

The implementation uses onChange, onSubmit, and onCellEdit Client Scripts alongside UI Policy Actions so that Assignment Group and Urgency respond appropriately to Impact and State changes are controlled according to the defined rules. The solution is lightweight and suitable for a short-duration micro project while demonstrating practical techniques for improving form usability and data integrity.

12. Future Enhancement

- Extend UI Policies and Client Scripts to cover additional Incident scenarios.
- Introduce advanced validation rules for critical Incident information.
- Add role-based controls for selected Incident fields and actions.
- Integrate automated notifications for important Incident changes.
- Develop enhanced dashboards and reports for Incident monitoring.
- Add further automation for repetitive Incident management activities.
- Improve contextual messages and user guidance.

Final project documentation should be supported with the relevant ServiceNow configuration and testing screenshots. The supplied project materials identify screenshots for UI Policy, UI Policy Actions, Client Scripts, and the five testing activities.